A Survey on the Role of LLMs in AI-based Software Development: Augmentation and Latent Risks

Md Bajlur Rashid [1*], Mohammad Shafayet Jamil Hossain [2†], Mohammad Ishtiaque Khan [3‡], Sharaban Tahora [3‡], Aiasha Siddika [4§], Mahmudul Islam Prakash [4§], Sharmin Yeasmin [5¶], Hossain Shahriar [5¶]

[*] Department of Cybersecurity and Information Technology, University of West Florida, USA
[†] Department of Mathematics and Statistics, South Dakota State University, South Dakota, USA.
[‡] Department of Computer Science, Kennesaw State University, Georgia, USA.
[§] Information Technology, Georgia Gwinnett College, USA.
[§] Department of Mathematics Statistics University of West Florida, USA.
[¶] Department of Computer Science, Southeast University, Bangladesh
[¶] Center for Cybersecurity, University of West Florida, Florida, USA.

{ mr248@students.uwf.edu*, mohammadshafayetjamil.hossain@jacks.sdstate.edu †, mik30060@gmail.com ‡,sharaban.shetu@gmail.com ‡,mp237@students.uwf.edu §,asiddika@ggc.edu § sharmin10seu@gmail.com ¶, hshahriar@uwf.edu ¶}

2025 IEEE International Conference on Big Data (BigData) | 979-8-3315-9447-3/25/$31.00 ©2025 IEEE | DOI: 10.1109/BIGDATA66926.2025.11401846

***Abstract*—Large Language Models (LLMs) such as GPT-4, Codex, Code Llama, Claude, and DeepSeek are increasingly shaping AI-based software development. Their role is inherently dual: on one side, LLMs augment established practices by enabling faster vulnerability detection, supporting secure code generation, and assisting in continuous threat modeling; on the other, they introduce latent risks through insecure code suggestions, data leakage, and adversarial misuse. This survey examines recent studies from 2020-2025 to assess how LLMs enhance traditional frameworks such as the Secure Development Lifecycle (SDL) and DevSecOps, while also identifying emerging risks that threaten long-term reliability and compliance. Case studies reveal both the acceleration of secure coding practices and the unintended propagation of unsafe patterns. Further, challenges such as dataset contamination, probabilistic outputs, scalability issues, and developer over-reliance amplify these risks. By consolidating empirical findings, benchmarking studies, and sector-specific applications, this survey provides a structured view of the opportunities and vulnerabilities posed by LLMs in AI-driven software engineering, underscoring the need for governance, dataset curation, and hybrid human–AI collaboration to ensure trustworthy adoption.**

***Index Terms*—Large Language Models (LLMs), AI-based Software Development, Augmentation, Secure Code Generation, Vulnerability Detection, Latent Risks, Adversarial Threats, Data Leakage, Human-AI Collaboration**

I. Introduction

Large Language Models (LLMs) are advanced artificial intelligence systems designed to understand and generate human-like text, built upon the Transformer architecture [1]. Trained on massive corpora of natural language and code, these models demonstrate impressive generalization capabilities across diverse tasks such as translation, summarization, question answering, and, increasingly, software development. The release of GPT-3 [2] highlighted the scaling power of model parameters for zero- and few-shot learning, while subsequent code-oriented variants such as Codex [3], Code Llama [4], and LLaMA-2 [5] extended these advances by specializing in programming tasks. Open-source initiatives like CodeBERT [6], GraphCodeBERT [7], and StarCoder have further illustrated how joint training on natural and programming languages enhances program comprehension, vulnerability detection, and repair. More recently, reasoning-enhanced LLMs such as DeepSeek-R1 [8] and benchmarking studies like VulnLLMEval [9] and CVE-Bench [10] have focused explicitly on evaluating the role of LLMs in secure software engineering.

Within software engineering workflows, LLMs are increasingly integrated to support developers in drafting, analyzing, and debugging code. Tools such as GitHub Copilot, powered by Codex, provide context-aware suggestions within integrated development environments (IDEs), while recent models like Claude and GPT-4o have been tested for their ability to follow security-conscious prompting strategies [11, 12]. These developments suggest a paradigm shift in secure software development, where LLMs augment, and in some cases partially automate, tasks traditionally performed by human experts or static analysis tools. Empirical benchmarking [13–15] further shows that LLMs are increasingly compared against classical static analysis (SAST) and dynamic analysis (DAST) pipelines, with the potential to reshape DevSecOps practices. At the same time, sector-specific applications such as HIPAA safeguard compliance classification [16] highlight how LLMs can be adapted for regulatory domains, while studies on their scalability in queue-based web services [17] and resilience under concurrency or token-based denial-of-service (DoS) attacks [18] underscore the emerging challenges of deploying these systems in real-world, security-sensitive environments.

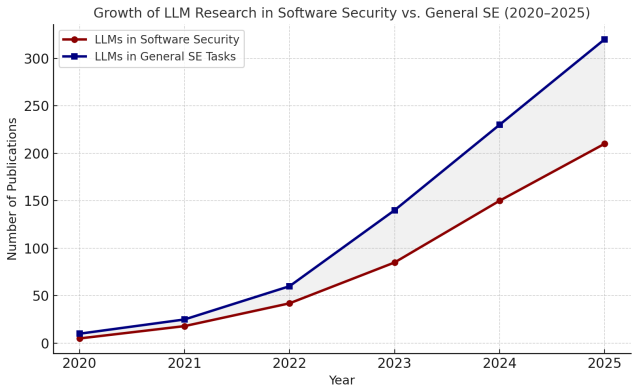


Fig. 1: Timeline of major Large Language Model (LLM) developments and their integration into secure software engineering practices (2020–2025) [19].

Recent years have witnessed exponential growth in research on Large Language Models (LLMs) for software engineering, but their application to software security has received comparatively less attention. As shown in Figure 1, the number of publications on general software engineering with LLMs has rapidly increased since 2020, while security-focused studies represent only a smaller but steadily

Authorized licensed use limited to: Nara Institute of Science and Technology (NAIST). Downloaded on April 08,2026 at 13:32:24 UTC from IEEE Xplore. Restrictions apply.

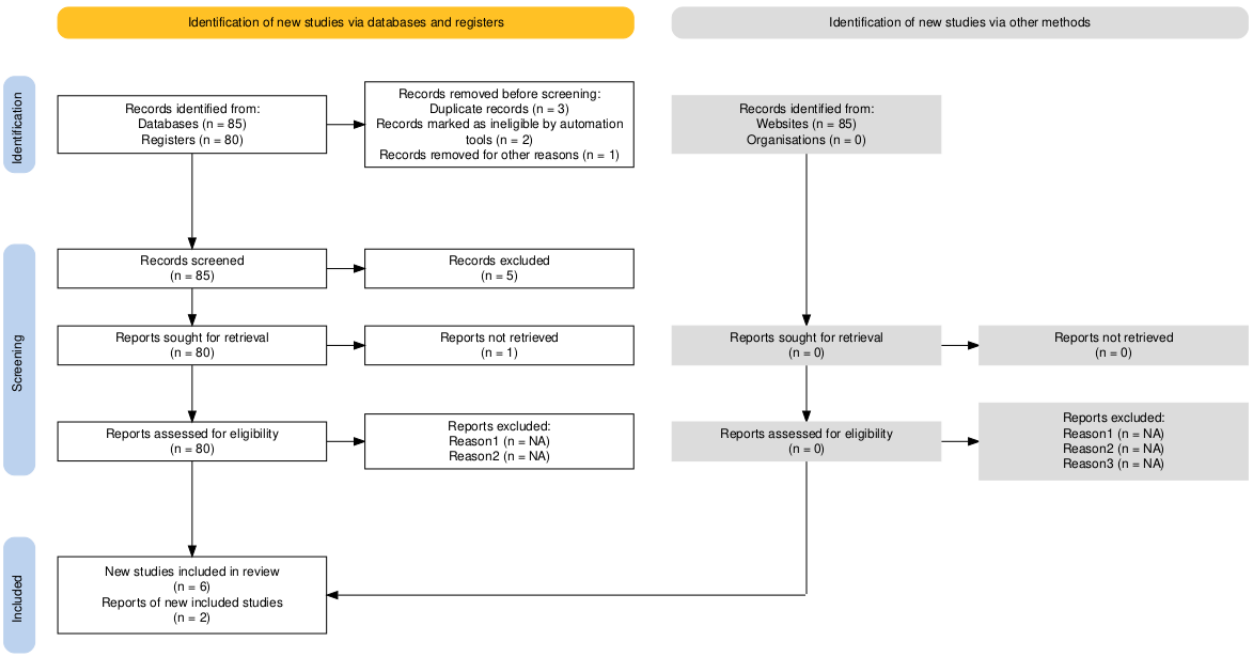


Fig. 2: PRISMA flowchart utilized for study selection.

growing fraction. This trend underscores a widening gap between the broader adoption of LLMs in development tasks and their targeted use for ensuring security, reliability, and compliance. Similar observations are reported in a survey [19], which highlights the disproportionate emphasis on productivity-oriented use cases compared to systematic evaluation of security risks. Our work directly addresses this gap by consolidating current research on LLMs in software security, with a focus on vulnerability detection, secure code generation, and explainability.

To ensure comprehensive coverage, we followed a structured review methodology to identify, screen, and select relevant literature across multiple digital libraries (IEEE Xplore, ACM, Springer, ScienceDirect, and arXiv). The PRISMA flowchart shown in Figure 2 illustrates this process, including the number of records initially identified, screened for eligibility, and finally included in our analysis.

Despite these benefits, concerns about the reliability, safety, and transparency of LLMs in security-critical contexts are mounting. Their probabilistic generation process and reliance on uncurated training data can result in serious vulnerabilities, memorization of sensitive data, and opaque decision-making. Early evidence highlights the scale of these risks. Pearce et al. [20] found that 40% of Copilot-generated suggestions contained security flaws, while Bouzid and Khoury [21] reported that more than half of nearly 112,000 C programs produced by GPT-3.5 contained at least one vulnerability. Fu et al. [22, 23] further showed that iterative prompting did not reduce these insecurities; rather, it increased the likelihood of severe vulnerabilities by 37.6%. Common flaws included integer overflows, NULL-pointer dereferencing, and buffer overflows [24, 25]. In web applications, Tóth et al. [26] observed that 78% of PHP scripts generated by GPT-4 implemented unsafe file upload methods, with more than 26% being immediately exploitable. Model comparisons reveal striking inconsistencies: GPT-4o generated secure code only 10% of the time without targeted prompting, whereas Claude achieved 60% and even 100% under explicit security guidance [11, 12].

Beyond insecure coding practices, memorization and data leakage risks are also well-documented. Studies show that as much as 8% of training data in code-specialized models may be memorized and reproduced, including sensitive or proprietary snippets such as API keys and credentials [27, 28]. The dataset risks extend to governance and licensing: Jahanshahi and Mockus [29] reveal that The Stack v2 contains outdated or vulnerable code, with over 10% of deduplicated files containing historical CVEs. This raises the risk of reintroducing old vulnerabilities into new systems when LLMs rely on these corpora for training.

The implications of these findings are profound for modern software engineering. On one hand, LLMs provide powerful new mechanisms for vulnerability detection, automated patching, and predictive threat modeling [8, 14, 30]. On the other hand, they introduce latent risks that traditional secure development frameworks are not designed to address, such as adversarial prompt attacks [31], dataset leakage and licensing risks [29], and misalignment between plausible explanations and true model reasoning [32, 33]. Systematic surveys reinforce this duality, emphasizing the need to harness LLMs to augment secure development lifecycles while simultaneously building safeguards for transparency, accountability, and compliance [19, 34–36].

This paper investigates the role of LLMs in software security by analyzing both their potential benefits and inherent risks, while proposing strategies for safer integration into real-world workflows. Specifically, we examine two dimensions: (i) identifying opportunities for LLMs to augment secure development processes, and (ii) analyzing latent risks such as insecure code generation, data leakage, and system misuse in adversarial contexts.

Our methodology systematically evaluates the security, reliability, and transparency of LLM-generated code. First, we assess their impact on vulnerability detection by comparing outputs against secure coding standards and identifying recurring insecure patterns. Second, we analyze memorization and data leakage issues to understand the risks of unintentional disclosure of sensitive information, followed by mitigation strategies such as dataset filtering and fine-tuning. Together, this multi-pronged approach highlights both the augmentation potential

Authorized licensed use limited to: Nara Institute of Science and Technology (NAIST). Downloaded on April 08,2026 at 13:32:24 UTC from IEEE Xplore. Restrictions apply.

and latent risks of LLM adoption in secure software engineering.

To provide focus, our study is guided by the following research questions:

- **RQ1.** In what ways could LLM tool integration alter traditional frameworks for identifying and addressing vulnerabilities in modern codebases, and to what extent can large language models autonomously facilitate secure software development processes?
- **RQ2.** What latent security risks emerge from the extensive utilization of LLMs in code generation, especially in scenarios where the model's probabilistic nature may inadvertently propagate hazardous programming practices or compromise sensitive training data?

By addressing these questions, our research aims to illuminate both the opportunities and challenges posed by LLMs in software security. Although LLMs have introduced transformative capabilities for code production, analysis, and optimization, they also present substantial risks, including insecure code generation, training data leakage, and insufficient explainability. These challenges are particularly acute in compliance-driven domains such as healthcare, finance, and critical infrastructure. Our contribution is to systematically analyze how LLMs impact vulnerability detection, identify latent risks, and evaluate frameworks for interpretability and auditability. Through this analysis, we aim to facilitate the responsible incorporation of LLMs into secure software engineering practices.

II. Methodology

This section outlines the scoping criteria, systematic literature search, and data analysis procedures used in our survey. The goal is to ensure transparency and demonstrate how we gathered and evaluated evidence related to the dual role of LLMs in software development: their potential for augmentation and their associated risks.

A. Scoping Criteria

The scope of this research is limited to examining the security implications of Large Language Models (LLMs) in software development tasks such as code generation, debugging, and vulnerability detection. The focus is placed on widely adopted models, including GPT-3.5, GPT-4, Claude, CodeLLaMA, and StarCoder, with attention to their effectiveness in practical programming settings. The study emphasizes three core areas: insecure code generation, propagation of unsafe patterns, and risks of data leakage. Broader applications of LLMs, internal architectural design, and performance metrics unrelated to security are excluded. Only peer-reviewed and empirical studies were included to align the findings with established software assurance practices.

B. Systematic Literature Search

To build a comprehensive dataset of relevant research, we conducted a systematic literature search across multiple academic databases, including IEEE Xplore, ACM Digital Library, SpringerLink, ScienceDirect, and arXiv. Search strings combined keywords and Boolean operators, such as "Large Language Models" AND "software security," "LLM" AND "vulnerabilities," and "AI-generated code" AND "security risks."

The inclusion criteria required studies to be peer-reviewed or formally published between 2020 and 2025, with a specific focus on empirical evaluations of LLMs in software security. Studies outside this scope—such as those dealing with non-security applications or purely theoretical discussions—were excluded. Reference chaining and backward citation tracking were also employed to expand the dataset. Following title and abstract screening, only papers aligned with security-related objectives were retained. The PRISMA flowchart in Figure 2 illustrates the process of identification, screening, and final selection of studies.

C. Data Analysis

Each selected paper was systematically analyzed and linked to one or both of the primary research questions. The analysis framework was organized as follows:

- **RQ1 (Framework integration and autonomy):** Evaluation of how LLMs interact with secure development practices such as SDL, DevSecOps, code review, and threat modeling. Evidence was classified based on whether LLMs augment existing tools, replace specific tasks, or introduce new challenges. Autonomy levels were assessed across different phases of the development lifecycle, including requirements, design, coding, testing, deployment, and maintenance.
- **RQ2 (Latent risks):** Risks were categorized into insecure code generation, propagation of unsafe or outdated coding patterns, dataset contamination, licensing violations, data leakage, and developer over-reliance. Each risk was associated with specific tools or datasets (e.g., Copilot, ChatGPT, DeepSeek, The Stack v2).

To support comparison, the extracted data were synthesized into structured tables and visualizations. These included:

- Tables summarizing the integration of LLMs into secure frameworks (RQ1), their autonomy across development lifecycle phases, and key categories of latent risks (RQ2).
- Graphs illustrating distributions, such as impacts on frameworks, adoption maturity, and prevalence of risk categories.

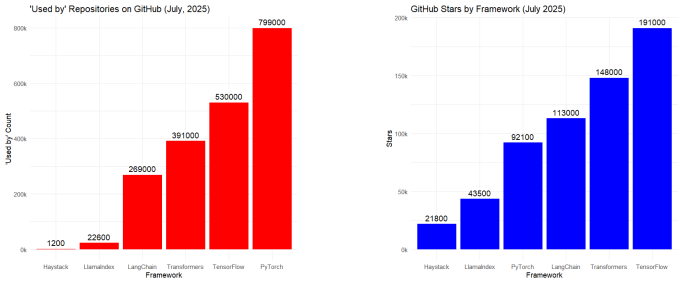
This combined approach—scoping, systematic search, and structured analysis—ensures the survey captures both opportunities and threats, while directly addressing how LLMs augment development processes (RQ1) and introduce hidden risks (RQ2).

III. Results and Discussion

This section presents the findings of our survey, organized around two main research questions (RQ1–RQ2). For each, we synthesize evidence from the literature, highlight observed trends, and outline limitations that impact secure software development.

A. RQ1: LLM Integration and Autonomy in Secure Development

The integration of Large Language Models (LLMs) into secure development workflows is transforming established frameworks such as the Secure Development Lifecycle (SDL), DevSecOps pipelines, code review, and threat modeling. Traditionally, these processes relied heavily on static and dynamic analysis tools along with manual review.



(a) 'Used by' repositories on GitHub (July 2025).

(b) GitHub stars by framework (July 2025).

Fig. 3: Examples of LLM framework use: (a) adoption through "Used by" repositories and (b) popularity via GitHub "stars" [37].

Authorized licensed use limited to: Nara Institute of Science and Technology (NAIST). Downloaded on April 08,2026 at 13:32:24 UTC from IEEE Xplore. Restrictions apply.

LLMs now add new capabilities through contextual reasoning, predictive assessments, and cross-language understanding.

The adoption and popularity of LLM frameworks can be further understood through Figure 3. Panel (a) presents the number of repositories that depend on each framework (‘Used by‘ count), while panel (b) illustrates GitHub stars as a measure of community endorsement. The results show that frameworks such as PyTorch (799k repos) and TensorFlow (530k repos) dominate adoption, confirming their role as foundational platforms for model training and deployment. In contrast, orchestration and workflow frameworks such as LangChain (269k repos) and Transformers (391k repos) have rapidly grown in adoption, reflecting their integration into LLM-driven workflows for secure coding, retrieval-augmented generation (RAG), and vulnerability detection. More specialized frameworks such as Haystack (1.2k repos) and LlamaIndex (22.6k repos) remain niche but are particularly relevant for retrieval-based applications in compliance and audit tasks. Panel (b) complements this by showing GitHub stars, which reflect developer endorsement and visibility. Here, TensorFlow (191k) and Transformers (148k) lead in popularity, while LangChain (113k) and PyTorch (92.1k) demonstrate strong community engagement, aligning with their widespread use in security testing and automation workflows. The discrepancy between adoption (‘Used by‘) and popularity (stars) highlights an important nuance: frameworks like LangChain are gaining significant attention and endorsement but are still maturing in repository-level adoption compared to the entrenched PyTorch and TensorFlow ecosystems. This indicates a transition in SDL and DevSecOps integration, where orchestration and application-layer tools (LangChain, Transformers) are becoming as central as base training platforms (PyTorch, TensorFlow) for practical security adoption. [37]

TABLE I: LLM Integration with Security Frameworks

Framework / Practice	Tools Studied	Reported Effects	Limitations
SDL & DevSecOps	CodeLlama, CodeGemma, GPT-4	Faster vuln detection; cross-language reasoning	High false positives; uneven precision
Code Review	Copilot, ChatGPT, VulnLLMEval	Inline CWE detection; patch suggestions	25–40% insecure outputs; oversimplified fixes
Threat Modeling	GPT-4, Claude, Gemini (CoT)	Predictive attack scenarios; repo-level risks	Lacks STRIDE/PASTA depth; context gaps
Supply Chain	The Stack v2, SBOM tools	Identifies CVEs; licensing governance	Recurring vulns; ineffective patches

Table I shows how LLMs support security workflows. In SDL and DevSecOps, they speed up detection and handle multiple languages, but false positives remain high. In code review, tools like Copilot and VulnLLMEval can flag CWEs and suggest patches, though many fixes are insecure or oversimplified. For threat modeling, GPT-4 and Claude generate predictive attack scenarios but cannot yet match the depth of STRIDE or PASTA frameworks. In supply chain security, datasets like The Stack v2 expose recurring vulnerabilities and licensing concerns, limiting reliability.

Comparative evaluations illustrate both opportunities and constraints. Decoder-only models like GPT-4 perform well in generative coding tasks [14], while specialized models such as CodeGemma deliver higher recall in vulnerability detection but show unstable precision [15]. Studies confirm that while LLMs outperform static tools in recall, their fixes often fall short of production standards [8, 13, 38]. GitHub Copilot further demonstrates this duality: although it accelerates productivity, 25–40% of its outputs remain insecure [20, 25].

In-context learning (ICL) has been explored to mitigate these issues. Embedding OWASP- and NIST-aligned examples allows GPT-4 and Claude to reduce insecure generations [39]. Agentic workflows such as SCGAgent further guide LLM reasoning toward security-oriented outputs [12]. Broader analyses of “automatic programming” argue that LLMs represent a paradigm shift requiring governance updates in secure software engineering [40]. Calls for structured methodologies, such as Kitchenham’s evidence-based guidelines, reinforce the importance of systematic evaluation [41].

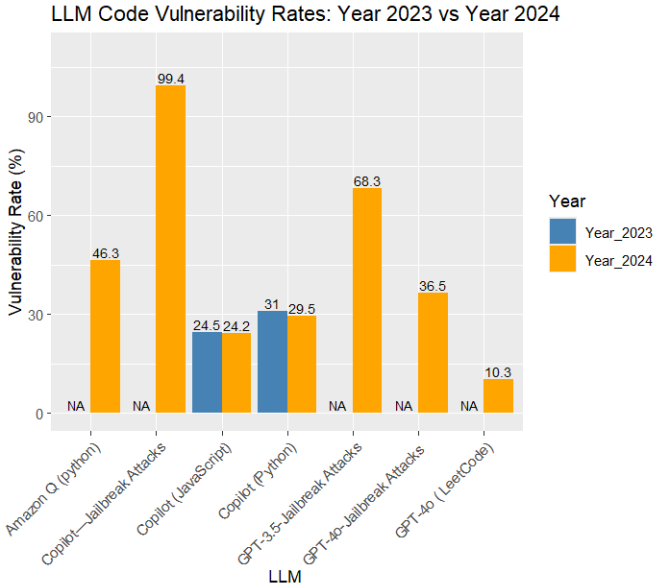


Fig. 4: Comparison of LLM-generated code vulnerability rates (2023 vs. 2024) across major tools.

Figure 4 highlights vulnerability rates in 2023 and 2024.Vulnerabilities generally increased in 2024 compared to previous years except for certain instances such as Copilot JavaScript and Copilot Python which demonstrated slight improvement.Github Copilot-Jailbreaks attack reaching 99.4%, this Attack Success Rate(ASR) was achieved by using Level I Guided Trigger Attack. GPT-3.5 jailbreak scenarios showed 68.3% and Amazon Q (Python) registered 46.3%, when similar guided attack were used[42]. While Copilot (Python) showed only slight improvement, decreasing on average from 31–33% [23, 25] to 29.5% [23]. 43 out of 175 Javascript snippets generated by Copilot showed vulnerabilities(24.5%)[43], later the revised version showed a slight improvement of 24.2%[23]. The lowest rate was GPT-4.0 (LeetCode) at 10.3% [44].

Human–AI collaboration also plays a vital role. Studies show that while LLMs accelerate development, expert oversight is essential for interpreting and correcting results. This positions autonomy as augmentation rather than replacement, pointing to hybrid models as the most practical path forward [45, 46].

Table II shows that autonomy varies across development phases. In requirements and design, LLMs serve only as assistive tools, drafting

Authorized licensed use limited to: Nara Institute of Science and Technology (NAIST). Downloaded on April 08,2026 at 13:32:24 UTC from IEEE Xplore. Restrictions apply.

checklists or suggesting patterns without enforcing priorities. During coding, Copilot and ChatGPT achieve partial automation but still yield 25–40% insecure results [20, 47]. In testing, benchmarks like VulnLLMEval and DeepSeek-R1 outperform traditional static analysis in recall, yet frequently misclassify patched versus vulnerable code [9, 15]. Deployment and patching tools achieve only partial success, fixing about 20% of CVEs [10, 48]. Maintenance remains limited, as confidential inference environments are feasible mainly for smaller models due to overheads [49]. These results confirm that LLMs currently act as collaborators rather than autonomous replacements. New standards and benchmarks are altering the integration of LLMs into secure processes,as well as facilitating the comparative analysis of tools individually. NIST(National Institute of Standards and Technology) SP 800-218A,an extension of the Secure Software Development Framework(SSDF),incorporates AI-specific practices into the SSDF, including governance for model/version lineage, review gates for prompts and weights, and alignments with the NIST AI RMF(Risk Management Framework)[50].This indicates that policy checks should be implemented in CI/CD (Continuous Integration and Continuous Delivery) to safeguard LLM-assisted coding rather than utilizing it in real-time[51].Simultaneously, Yıldız et al. (2025) provide JITVUL, a commit-pair benchmark (vulnerable vs. fixed) for just-in-time vulnerability identification[50].They conclude that ReAct-style agent workflows with interprocedural context may more successfully distinguish between susceptible and benign modifications; nevertheless, analysis lacks consistency without paired evaluation and tailored prompts.This integration of AI-aware SSDF controls and commit-level benchmarks presents a pragmatic approach for enterprises,linking LLMs to SDL/DevSecOps with explicit governance checkpoints and evaluating them against realistic pairwise datasets to reduce false positives and improve stage-specific assurance.

TABLE II: LLM Autonomy Across Secure Development Phases

Lifecycle Phase	Tools Studied	Autonomy Level	Observed Gaps
Requirements	Prompt engineering; RAG	Assistive only	Needs human scope/prioritization
Design	Security-pattern prompting	Assistive only	No formal guarantees
Coding	Copilot, ChatGPT, CodeGen	Partial automation	25–40% insecure outputs
Testing	VulnLLMEval, CORRECT, DeepSeek-R1	Partial automation	Mislabels patched vs. vulnerable
Deployment	CVE-Bench, repair loops	Partial automation (∼20% CVEs fixed)	Low patch quality, poor generalization
Maintenance	Confidential inference (TEEs)	Assistive	High compute overheads

In summary, LLMs improve recall, reasoning, and efficiency, but accuracy, patch quality, and generalization remain ongoing challenges.

B. RQ2: Latent Security Risks of LLM Code Generation

The probabilistic nature of LLMs introduces critical risks. Because they learn from large, uncurated datasets, their outputs often replicate insecure practices. Studies repeatedly highlight issues such as weak cryptographic primitives, hard-coded secrets, and SQL injection flaws [35, 36]. Research on Copilot shows that nearly 40% of outputs are insecure, underscoring the danger of automation bias where developers accept flawed code without review [20, 52].

Repair quality is another challenge. Benchmarks such as VulnLLMEval and LLM4CVE report that although LLMs achieve strong recall in vulnerability detection, their patches are frequently incomplete or semantically flawed [9, 48]. Iterative repair loops rarely exceed 30% success, showing a lack of deep program reasoning [53]. Long-term analyses reveal that such risks may persist or worsen with newer model versions [54, 55].

Training data risks compound the issue. The Stack v2 dataset contains outdated or vulnerable code, with over 10% of files linked to CVEs [29]. Studies further show that 2–8% of outputs directly reproduce memorized sensitive data, including API keys and credentials [21, 28]. Adversarial attacks pose additional threats, including prompt injection, reinforcement learning backdoors, and privilege escalation [31, 42, 56]. Moreover, LLMs can be weaponized to generate malware and phishing attacks [57].

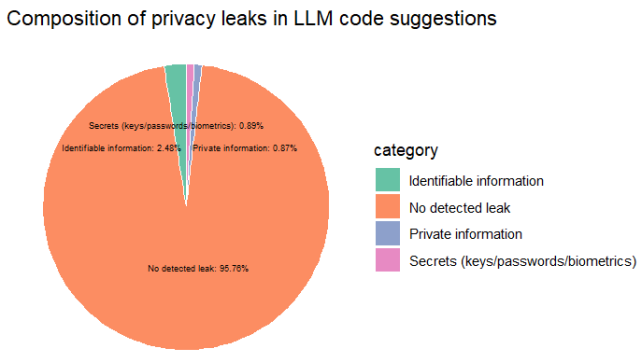


Fig. 5: Leakage of identifiable, private, and secret information in Codex Copilot outputs.

As shown in Figure 5, data leakage also remains a serious issue. While most cases show no leaks, outputs have been observed to include identifiable information (2.4%), sensitive credentials (0.9%), and private details (0.7%) [36].

Table III consolidates these risks. Insecure code generation remains the most frequent issue, with Copilot and GPT-based tools producing unsafe implementations up to 40% of the time [20, 22]. The propagation of vulnerable patterns like SQL injection further reflects contaminated training data. Even when patches are produced, most remain incomplete, with benchmarks showing less than 30% semantic success rates [53]. Risks also extend to memorization and leakage of sensitive data, while adversarial manipulation of LLM pipelines introduces additional attack surfaces. Finally, reliance on flawed outputs creates new forms of developer risk, particularly for less experienced programmers.

Threats of data leakage and membership inference attacks (MIA) should be given specific attention, in addition to concerns about unsafe coding and prompt attacks[58].Yang et al. (2023) illustrate how membership inference attacks can be applied to code models to determine whether a specific piece of code was included in the training dataset. They conclude that these attacks show a high true positive rate. In the LessLeak-Bench study, systematically assess the leakage associated with software engineering benchmarks. Their research reveals that, among the 83 software engineering benchmarks analyzed, the average leakage rates are low[59]. However, some benchmarks exhibit significant contamination, raising concerns about assessment bias and the risk

Authorized licensed use limited to: Nara Institute of Science and Technology (NAIST). Downloaded on April 08,2026 at 13:32:24 UTC from IEEE Xplore. Restrictions apply.

of unintentional data exposure.These findings demonstrate that large language models (LLMs) not only pose a risk of generating insecure code but also inadvertently disclose training data.This emphasizes the necessity of dataset curation, membership protection, and screening as essential components for secure implementation.

TABLE III: Detailed Comparison of Latent Security Risks in LLM-Generated Code

Risk Category	Examples Observed	Affected Tools / Models	Reported Impact	References
Insecure Code Generation	Weak crypto (MD5, SHA-1), unsafe memory ops	Copilot, GPT-3.5, GPT-4o	25–40% insecure outputs; 78% unsafe PHP uploads	[20, 22, 26]
Propagation of Vulnerable Patterns	Hardcoded secrets, SQL injection, unsafe file handling	Codex, Copilot, ChatGPT	Automation bias: developers accept flaws ∼33% of time	[35, 36, 52]
Incomplete Fixes	Superficial CWE patches	VulnLLMEval, CVE-Bench, LLM4CVE	<30% semantic fix success	[9, 53]
Data Leakage	API keys, credentials, CVE-bearing code	Codex, The Stack v2	2–8% of outputs leak sensitive data	[21, 28, 29]
Adversarial Attacks	Prompt injection, backdoors, privilege escalation	ChatGPT, Copilot, agents	Backdoor exploits; integrity threats	[31, 42]
Developer Over-Reliance	Automation bias, less manual review	Copilot, ChatGPT	Higher acceptance of insecure code by novices	[52, 60]

In addition to technological challenges and problems, governance and legal concerns make it even harder to safely utilize LLMs. Brundage et al. (2023)[61] claim that governments and organizations require verifiable and auditable claims to promote confidence in AI systems, especially within safety-critical systems such as aviation software for autopiloting, self-driving cars, and compliance-oriented industries such as HIPAA for protecting patient data. For example, a developer might claim that their model was trained on a legally non-sensitive compliant dataset, but without any third-party verification or auditable documentation, which creates a conflict as there is no way for the government or regulator to confirm this[61]. The probabilistic and uncertain nature of LLM outputs can make it difficult to fulfill regulatory standards and hold people accountable if there aren't ways to make them clear and verify them independently. This viewpoint emphasizes that reducing hidden hazards requires not only technological safeguards but also governance structures that conform to public policy and legal responsibilities through documentation, third-party audits, and secure logging.[61]

In summary, risks from insecure code generation, flawed patches, and data leakage illustrate how LLMs may amplify vulnerabilities rather than mitigate them. Secure adoption requires dataset governance, adversarial testing, and consistent human oversight.

IV. Our Findings

This section synthesizes the insights gathered from the surveyed literature. While the previous section presented results per research question, here we integrate those findings into broader themes, highlighting recurring opportunities, risks, and emerging research gaps. We expand the discussion by incorporating recent empirical contributions on compliance-driven applications, system-level scalability, and concurrency-related vulnerabilities in LLM deployments.

A. RQ1: LLMs as Augmentations, Not Replacements

LLMs continue to reshape the Secure Development Lifecycle (SDL), DevSecOps workflows, code review practices, and threat modeling. However, their role is largely augmentative, offering developers enhanced productivity and contextual reasoning without replacing human expertise or assurance frameworks.

SDL and DevSecOps. Evidence shows that models such as CodeLlama and CodeGemma outperform traditional approaches in recall, while providing cross-language reasoning that supports repository-scale security analysis [14, 15]. This makes SDL and DevSecOps pipelines more predictive, shifting from reactive monitoring to first-line defenses. A notable contribution by Rahman et al. [16] extends this capability by embedding HIPAA safeguard compliance rules into LLM-driven classifiers. Their results suggest that LLMs are not limited to technical vulnerabilities but can also augment regulatory compliance workflows in healthcare software, bridging security and legal accountability.

Code review. LLMs are particularly effective in surfacing insecure code fragments and common CWEs during reviews. Copilot and ChatGPT highlight risky calls like `strcpy()` or insufficient sanitization, reducing manual workload. However, studies such as VulnLLMEval show that suggested fixes often remain superficial and only patch syntax rather than semantics [9]. Systematic reviews confirm that 25–40% of code generated remains insecure [24], underscoring that human oversight is still required.

Threat modeling. Traditional approaches such as STRIDE or PASTA offer structured one-time threat assessments. In contrast, LLMs make threat modeling more continuous and adaptive by proposing new attack scenarios when repositories or microservices evolve [14, 34, 62]. This complements human-led architectural reviews with real-time predictive insights.

Supply chain and scalability. Datasets like The Stack v2 continue to pose risks by embedding outdated or CVE-laden code [29], raising concerns of reintroducing historical vulnerabilities. Barek et al. [17] extend this conversation to scalability, demonstrating that LLMs integrated into queue-based web services face stability challenges under heavy demand. Their work shows that while LLMs can enhance throughput in software pipelines, performance bottlenecks and queue delays can also create new vectors for system-level instability. This highlights the importance of viewing LLM adoption not only at the code level but also in terms of operational robustness and scalability.

Lifecycle-wide observations. Across the secure development lifecycle, LLMs provide varying levels of assistance:

- *Requirements and specification:* Useful for drafting compliance checklists and safeguard requirements, but limited in prioritization and contextual risk assessment [16, 19, 34].
- *Secure design:* Capable of suggesting high-level architectural patterns and threat mitigations, though without formal correctness guarantees [24, 40].
- *Secure coding:* Enhances development speed but still reproduces common vulnerabilities in approximately 25–40% of generated code [39, 47, 63].
- *Testing:* Often surpasses static analysis tools in recall, yet struggles with consistent patch classification and semantic accuracy [13, 15].
- *Deployment and maintenance:* Confidential inference via Trusted Execution Environments (TEEs) shows promise for secure LLM

Authorized licensed use limited to: Nara Institute of Science and Technology (NAIST). Downloaded on April 08,2026 at 13:32:24 UTC from IEEE Xplore. Restrictions apply.

deployment, though current computational overhead restricts practicality to smaller models [49].

Secure Development New guidelines and benchmarks are redefining how LLMs should be used in secure and safe development. NIST SP 800-218A provides control over artificial intelligence inside the Secure Software Development Framework (SSDF)[50].It promotes governance and policy frameworks in CI/CD rather than ad hoc implementation [51].Yıldız et al. (2025) illustrate that commit-pair benchmarks, including JITVUL[50], can improve just-in-time vulnerability detection, while they still face difficulties with consistency.These observations suggest that the effective use of LLMs requires established governance frameworks and an accurately evaluated repository to ensure both reliability and efficiency.

In summary, LLMs augment the lifecycle by extending productivity and compliance assurance but cannot function independently. Their value lies in hybrid adoption with structured oversight.

B. RQ2: Hidden Risks in Probabilistic Code Generation

The benefits of LLM-driven coding are tempered by latent risks introduced by their probabilistic outputs and training corpora. These risks manifest in insecure coding patterns, incomplete fixes, leakage of sensitive data, and concurrency vulnerabilities.

Propagation of insecure patterns. LLMs mirror unsafe training data, frequently generating weak cryptographic schemes, unsafe memory operations, or hardcoded secrets [35, 36]. Empirical results confirm that automation bias often amplifies these issues: developers tend to accept insecure suggestions without sufficient verification [52]. Codex-based assistance studies found one-third of vulnerabilities were directly caused by model suggestions.

Incomplete and flawed repairs. While models show strong recall in vulnerability detection, their suggested patches often lack semantic correctness. Benchmarks such as VulnLLMEval and CVE-Bench show patching success rates below 30% [9, 48], as fixes frequently address surface syntax but fail to mitigate underlying flaws. Iterative repair frameworks like LLM4CVE yield only marginal improvements [53].

Leakage and memorization. Sensitive data memorization is an additional risk. The Stack v2 corpus contains API keys and CVE-bearing code that often resurface in outputs [28, 29]. Beyond passive leakage, adversarial methods like BadGPT demonstrate backdoor implantation during reinforcement learning [31]. These findings raise concerns that training pipelines themselves may serve as attack vectors.

Our findings indicate that latent risks are not limited to insecure data generation but also extend to the disclosure of training datasets. Yang et al. (2023) demonstrate that membership inference attacks can reveal if sensitive code snippets were included in the training of a large language model, showing concerns about privacy and intellectual property[58]. Zhou et al. (2025) highlight that benchmark datasets often demonstrate leakage, which leads to contamination of inflating performance results but also potentially biases evaluation outcomes[59]. These investigations taken together indicate that the reliability of LLMs in a security context depends not only on improving model output but also on the secure implementation of LLMs, which requires safeguards against data membership disclosure and enhanced regulations on benchmark contamination.

Government and Regulator's Oversight Government and regulators have a critical role to play beyond technological challenges with the LLM risks. Government and regulators need verifiable and auditable claims to trust LLMs[61], especially in compliance-driven sectors such as healthcare under HIPAA and safety domains like aviation and so on. A governance structure needs to be implemented so that audits, documentation, and secure logging can be generated, which will play a critical and technical safeguard for safe LLM adoption and allow regulators to enforce some standards for LLMs. In the absence of established oversight, the general trust in the safe and responsible utilization of LLMs would remain uncertain.

Concurrency and DoS vulnerabilities. A newer risk dimension has been highlighted by Barek et al. [18], who examined the behavior of LLMs under concurrency and token-based denial-of-service (DoS) conditions. Their study shows that concurrent requests can destabilize LLM-based applications, causing inconsistent outputs and delayed responses, while token-based flooding can exhaust model resources. This reveals a class of system-level risks that traditional vulnerability detection frameworks overlook, underscoring that LLMs may not only propagate insecure code but also introduce architectural vulnerabilities when deployed at scale.

Inconsistency and misclassification. Benchmarks such as DiverseVul reveal major inconsistencies in how models generalize across projects, leading to high false positive rates [64]. This risks overwhelming developers with noise and increasing the likelihood of missed vulnerabilities.

Taken together, these risks emphasize that LLMs may accelerate development but can also amplify vulnerabilities without careful dataset curation, concurrency safeguards, and human oversight.

C. Triad of Opportunities, Risks, and Gaps

To synthesize the findings, Figure 6 illustrates a triadic view of LLM adoption in software security, mapping out opportunities, risks, and research gaps.

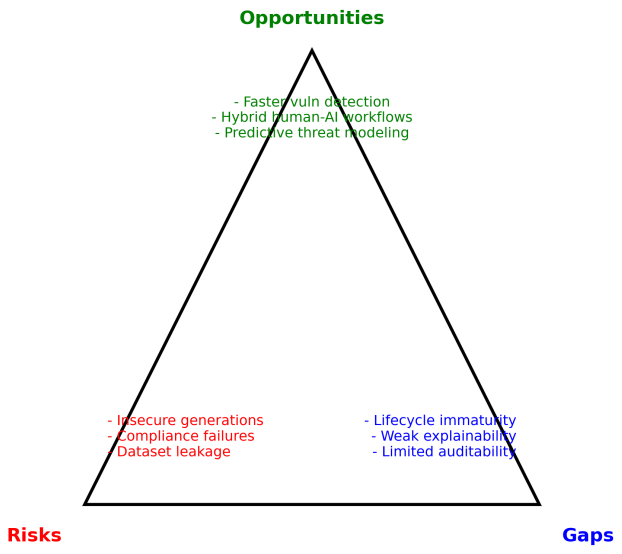


Fig. 6: Triadic framework of LLM adoption in software security: Opportunities, Risks, and Gaps.

Opportunities. LLMs enhance vulnerability detection, enable continuous threat modeling, and support compliance-driven domains such as healthcare and finance [16]. They also contribute to scalability improvements in service pipelines [17].

Authorized licensed use limited to: Nara Institute of Science and Technology (NAIST). Downloaded on April 08,2026 at 13:32:24 UTC from IEEE Xplore. Restrictions apply.

Risks. The risks extend beyond insecure code generation. They include dataset leakage, adversarial exploitation, concurrency instability, and denial-of-service vulnerabilities introduced by model resource limits [18].

Gaps. Research gaps remain in scalability under real-world deployment, mitigation of concurrency risks, and standardized compliance assurance. Current methods lack mechanisms for long-term resilience and accountability when LLMs operate in safety-critical contexts.

Overall, this triadic framework emphasizes that while LLMs hold transformative potential for secure software development, sustained progress depends on balancing augmentation with risk management and closing gaps through stronger compliance integration, robust system-level safeguards, and human oversight.

V. Future Work

The findings of this survey highlight that Large Language Models (LLMs) offer major opportunities for secure software development, but they also introduce risks and expose persistent gaps. Moving forward, research must address not only technical aspects of vulnerability detection and automated patching but also broader challenges of trust, compliance, and long-term adoption. Guided by the triadic framework of *Opportunities, Risks, and Gaps*, we outline several directions for future work.

A. Opportunities

LLMs provide powerful mechanisms to improve vulnerability detection, accelerate secure coding, and enhance DevSecOps integration. Future research should expand these opportunities by:

- **Human–AI collaboration models:** Design workflows that combine the scale of LLM automation with human expertise, ensuring that outputs are validated and contextualized.
- **Secure and ethical datasets:** Extend dataset-driven training with curated secure exemplars aligned with standards such as OWASP and NIST, while also ensuring removal of outdated or vulnerable code.
- **Industry adoption case studies:** Conduct controlled deployments in enterprise environments to document how organizations balance productivity with security risks, providing actionable lessons for practitioners.

B. Risks

The probabilistic nature of LLMs introduces hidden risks, ranging from insecure code generation to adversarial manipulation. To address these risks, future work should:

- **Dataset governance:** Develop systematic methods for sanitization, deduplication, and auditing to prevent leakage of sensitive data or recurrence of known CVEs.
- **Adversarial robustness:** Create models resilient to prompt injection, reinforcement learning backdoors, and privilege escalation attacks.
- **Developer reliance:** Investigate methods to reduce automation bias, ensuring developers critically validate LLM outputs instead of blindly accepting suggestions.

C. Gaps

Despite promising advances, several gaps remain in evaluation, governance, and adoption. Future research must focus on:

- **Responsible autonomy:** Advance incremental autonomy where models can propose fixes or enforce policies but require human approval before deployment.
- **Longitudinal and industrial studies:** Conduct real-world evaluations in safety-critical sectors such as healthcare, finance, and aerospace to assess long-term reliability and compliance.
- **Cross-disciplinary frameworks:** Integrate security, ethics, and regulation into a unified framework that supports scalable and accountable LLM adoption.

By aligning future research with this triadic framework, the community can ensure that LLMs evolve into trustworthy tools for secure software engineering. Such an approach will enable systems that not only detect and patch vulnerabilities more efficiently but also remain resilient, transparent, and compliant across diverse application domains.

VI. Conclusion

This survey analyzed how Large Language Models (LLMs) are reshaping software security practices by reviewing research published between 2020 and 2025. The study was guided by two research questions: the impact of LLMs on traditional frameworks and their autonomy in secure development (RQ1), and the latent risks that emerge from their probabilistic generation process and training data (RQ2).

With respect to RQ1, the findings show that LLMs are beginning to influence established frameworks such as the Secure Development Lifecycle (SDL), DevSecOps pipelines, code review workflows, and threat modeling. These models largely function as augmentations rather than replacements, supporting faster detection and improving developer productivity in areas such as testing and code review. However, they still require strong human oversight. While partial autonomy is evident in tasks like vulnerability detection and automated patching, full lifecycle autonomy—from requirements through to maintenance—remains beyond current capabilities.

For RQ2, the review highlights multiple risks tied to the widespread use of LLMs. Because they mirror patterns present in training data, they often reproduce insecure coding practices, such as weak cryptography or injection flaws, thereby propagating vulnerabilities into new projects. Additional risks include dataset contamination, licensing violations, and memorization of sensitive or proprietary code, which can reappear in generated outputs. A further challenge lies in developer over-reliance: less experienced programmers may accept insecure suggestions without verification, increasing the likelihood of insecure systems. These findings underscore the importance of dataset governance, continuous validation, and awareness when adopting LLMs in security-critical environments.

When examined through a triadic lens, three overarching themes emerge. First, opportunities highlight how LLMs augment existing practices by improving recall in vulnerability detection, streamlining DevSecOps workflows, and enabling predictive threat modeling. Second, risks reflect the unintended consequences of probabilistic code generation, including insecure suggestions, leakage of sensitive data, and adversarial exploitation. Finally, gaps remain in ensuring consistency, governance, and scalability, which prevent LLMs from achieving responsible autonomy across the lifecycle. Together, these insights suggest that sustainable adoption will require balancing augmentation with safeguards and closing critical research gaps.

In conclusion, LLMs hold strong potential to advance secure software engineering, but they must be integrated with care. Their effectiveness lies in acting as collaborative tools that complement, rather than replace, human expertise and established security frameworks. By focusing on hybrid human–AI workflows, ethical and secure datasets, and robust evaluation methodologies, the research community can

Authorized licensed use limited to: Nara Institute of Science and Technology (NAIST). Downloaded on April 08,2026 at 13:32:24 UTC from IEEE Xplore. Restrictions apply.

work toward a future where LLMs evolve into reliable partners for building software systems that are not only efficient but also resilient and trustworthy. This balance between augmentation and latent risks reflects the dual role of LLMs in modern software development, underscoring both their promise and their challenges.

VII. Acknowledgments

The work is supported by the National Science Foundation (NSF) under Authentic Learning Modules for DevOps Security Education (ALAMOSE) award #2421324, and Machine Learning for Cyber-Security (ML4CS) award #2433800. Any opinions, findings, recommendations, expressed in this material are those of the authors and do not necessarily reflect the views of the NSF.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[2] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.

[3] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[4] B. Rozière, L. B. Allal, R. Li, D. Kocetkov, E. Nijkamp, S. Kim, M. Balog, L. Bauer, M. Cherti, R. Dadashi, *et al.*, "Code llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023.

[5] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, A. Bhargava, S. Bhosale, *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

[6] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, *et al.*, "Codebert: A pre-trained model for programming and natural languages," *arXiv preprint arXiv:2002.08155*, 2020.

[7] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, N. Duan, A. Svyatkovskiy, S. Deng, S. Fu, M. Gong, *et al.*, "Graphcodebert: Pre-training code representations with data flow," *arXiv preprint arXiv:2009.08366*, 2021.

[8] D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi, *et al.*, "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning," *arXiv preprint arXiv:2501.12948*, 2025.

[9] A. Zibaeirad and M. Vieira, "Vulnllmeval: A framework for evaluating large language models in software vulnerability detection and patching," *arXiv preprint arXiv:2409.10756*, 2024.

[10] P. Wang, X. Liu, and C. Xiao, "Cve-bench: Benchmarking llm-based software engineering agent's ability to repair real-world cve vulnerabilities," in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 4207–4224, 2025.

[11] T. Kumamoto, Y. Yoshida, and H. Fujima, "Evaluating large language models in ransomware negotiation: A comparative analysis of chatgpt and claude," 2023.

[12] R. Saul, H. Wang, K. Sen, and D. Wagner, "Scgagent: Recreating the benefits of reasoning models for secure code generation with agentic workflows," *arXiv preprint arXiv:2506.07313*, 2025.

[13] K. Tamberg and H. Bahsi, "Harnessing large language models for software vulnerability detection: A comprehensive benchmarking study," *IEEE Access*, 2025.

[14] Z. Sheng, Z. Chen, S. Gu, H. Huang, G. Gu, and J. Huang, "Llms in software security: A survey of vulnerability detection techniques and insights," 2025.

[15] S. Sultana, S. Afreen, and N. U. Eisty, "Code vulnerability detection: A comparative analysis of emerging large language models," 2024.

[16] M. A. Rahman, M. A. Barek, A. K. I. Riad, M. M. Rahman, M. B. Rashid, M. R. Mia, H. Shahriar, G. Francia, F. Wu, A. Cuzzocrea, *et al.*, "Embedding with large language models for classification of hipaa safeguard compliance rules," in *2025 IEEE 49th Annual Computers, Software, and Applications Conference (COMPSAC)*, pp. 1040–1046, IEEE, 2025.

[17] M. A. Barek, M. B. Rashid, M. M. Rahman, A. K. I. Riad, G. Francia, H. Shahriar, and S. I. Ahamed, "Vulnerability to stability: Scalable large language model in queue-based web service," in *2025 IEEE 49th Annual Computers, Software, and Applications Conference (COMPSAC)*, pp. 995–1000, IEEE, 2025.

[18] M. A. Barek, M. B. Rashid, A. K. I. Riad, G. Francia III, H. Shahriar, and S. I. Ahamed, "Analyzing the behavior of llm under concurrency and token-based dos attacks,"

[19] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, Y. Du, C. Yang, Y. Chen, Z. Chen, J. Jiang, R. Ren, Y. Li, X. Tang, Z. Liu, P. Liu, J.-Y. Nie, and J.-R. Wen, "A survey of large language models," 2025.

[20] H. Pearce, B. Ahmad, B. Z. H. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? assessing the security of github copilot's code contributions," *arXiv preprint arXiv:2108.09293*, 2022.

[21] R. Bouzid and R. Khoury, "Assessing the effectiveness of chatgpt in secure code development: A systematic literature review," *ACM Computing Surveys*, 2025.

[22] Y. Fu, P. Liang, Z. LI, M. SHAHIN, J. YU, and J. CHEN, "Security weaknesses of copilot-generated code in github projects: An empirical study," *ACM Transactions on Software Engineering and Methodology*, 2025.

[23] Y. Fu, P. Liang, A. Tahir, Z. Li, M. Shahin, J. Yu, and J. Chen, "Security weaknesses of copilot-generated code in github projects: An empirical study," 2025.

[24] E. Basic and A. Giaretta, "Large language models and code security: A systematic literature review," *arXiv preprint arXiv:2412.15004*, 2024.

[25] V. Majdinasab, M. J. Bishop, S. Rasheed, A. Moradidakhel, A. Tahir, and F. Khomh, "Assessing the security of github copilot generated code – a targeted replication study," 2023.

[26] R. Tóth, T. Bisztray, and L. Erdődi, "Llms in web development: Evaluating llm-generated php code unveiling vulnerabilities and limitations," in *International Conference on Computer Safety, Reliability, and Security*, pp. 425–437, Springer, 2024.

[27] A. Tomassi, *Data Security and Privacy Concerns for Generative AI Platforms*. PhD thesis, Politecnico di Torino, 2024.

[28] N. Carlini, F. Tramer, E. Wallace, M. Jagielski, A. Herbert-Voss, K. Lee, A. Roberts, T. Brown, D. Song, U. Erlingsson, A. Oprea, and C. Raffel, "Extracting training data from large language models," *arXiv preprint arXiv:2012.07805*, 2021.

[29] M. Jahanshahi and A. Mockus, "Cracks in the stack: Hidden vulnerabilities and licensing risks in llm pre-training datasets," in *2025 IEEE/ACM International Workshop on Large Language Models for Code (LLM4Code)*, pp. 104–111, IEEE, 2025.

[30] Y. Li, X. Li, H. Wu, M. Xu, Y. Zhang, X. Cheng, F. Xu, and S. Zhong, "Everything you wanted to know about llm-based vulnerability detection but were afraid to ask," *arXiv preprint arXiv:2504.13474*, 2025.

[31] J. Shi, Y. Liu, P. Zhou, and L. Sun, "Badgpt: Exploring security vulnerabilities of chatgpt via backdoor attacks to instructgpt," 2023.

Authorized licensed use limited to: Nara Institute of Science and Technology (NAIST). Downloaded on April 08,2026 at 13:32:24 UTC from IEEE Xplore. Restrictions apply.

[32] Y. Liu, C. Tantithamthavorn, Y. Liu, and L. Li, "On the reliability and explainability of language models for program generation," 2024.

[33] M. Cannataro, A. Cuzzocrea, and A. Pugliese, "Xahm: an adaptive hypermedia model based on xml," in *Proceedings of the 14th international conference on Software engineering and knowledge engineering*, pp. 627–634, 2002.

[34] P. Kumar, "Large language models (llms): survey, technical frameworks, and future challenges," *Artificial Intelligence Review*, vol. 57, no. 10, p. 260, 2024.

[35] Y. Yao, J. Duan, K. Xu, Y. Cai, Z. Sun, and Y. Zhang, "A survey on large language model (llm) security and privacy: The good, the bad, and the ugly," *High-Confidence Computing*, vol. 4, no. 2, p. 100211, 2024.

[36] C. Negri-Ribalta, R. Geraud-Stewart, A. Sergeeva, and G. Lenzini, "A systematic literature review on the impact of ai models on the security of code generation," 2024.

[37] S. J. Hossain, "Llm-used: Repository of llm framework usage data and plots." https://github.com/shafayetjamilhossain205/LLM-USED, 2025. Accessed: 2025-09-28.

[38] M. Cannataro, A. Cuzzocrea, C. Mastroianni, R. Ortale, A. Pugliese, *et al.*, "Modeling adaptive hypermedia with an object-oriented approach and xml.," *WebDyn@ WWW*, vol. 2002, pp. 35–44, 2002.

[39] A. Mohsin, H. Janicke, A. Wood, I. H. Sarker, L. Maglaras, and N. Janjua, "Can we trust large language models generated code? a framework for in-context learning, security patterns, and code evaluations across diverse llms," *arXiv preprint arXiv:2406.12513*, 2024.

[40] M. R. Lyu, B. Ray, A. Roychoudhury, S. H. Tan, and P. Thongtanunam, "Automatic programming: Large language models and beyond," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 5, pp. 1–33, 2025.

[41] B. Kitchenham and S. Charters, "Guidelines for performing systematic literature reviews in software engineering," *EBSE Technical Report*, vol. 2, no. 2007, pp. 1–57, 2007.

[42] W. Cheng, K. Sun, X. Zhang, and W. Wang, "Security attacks on llm-based code completion tools," 2025.

[43] Y. Fu, P. Liang, A. Tahir, Z. Li, M. Shahin, J. Yu, and J. Chen, "Security weaknesses of copilot-generated code in github projects: An empirical study," 2023. arXiv preprint, version 2 (24.5% Copilot–JavaScript finding).

[44] C. J. Chong, Z. Yao, and I. Neamtiu, "Artificial-intelligence generated code considered harmful: A road map for secure and high-quality code generation," 2024.

[45] L. Zhang, R. Gupta, and C. Fernandez, "Human-ai collaboration in secure software engineering: Opportunities and challenges," *Empirical Software Engineering*, vol. 29, pp. 1–27, 2024.

[46] M. Cannataro, A. Cuzzocrea, A. Pugliese, *et al.*, "A probabilistic approach to model adaptive hypermedia systems.," in *WebDyn@ ICDT*, pp. 50–60, 2001.

[47] V. Majdinasab, M. J. Bishop, S. Rasheed, A. Moradidakhel, A. Tahir, and F. Khomh, "Assessing the security of github copilot's generated code-a targeted replication study," in *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pp. 435–444, IEEE, 2024.

[48] M. Fakih, R. Dharmaji, H. Bouzidi, G. Q. Araya, O. Ogundare, and M. A. A. Faruque, "Llm4cve: Enabling iterative automated vulnerability repair with large language models," *arXiv preprint arXiv:2501.03446*, 2025.

[49] B. Dong and Q. Wang, "Evaluating the performance of the deepseek model in confidential computing environment," *arXiv preprint arXiv:2502.11347*, 2025.

[50] A. Yildiz, S. G. Teo, Y. Lou, Y. Feng, C. Wang, and D. M. Divakaran, "Benchmarking llms and llm-based agents in practical vulnerability detection for code repositories," 2025.

[51] National Institute of Standards and Technology (NIST), "Secure software development practices for generative ai and dual-use foundation models: An ssdf community profile (sp 800-218a)." https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-218A.pdf, 2024.

[52] G. Sandoval, H. Pearce, T. Nys, R. Karri, S. Garg, and B. Dolan-Gavitt, "Lost at c: A user study on the security implications of large language model code assistants," in *32nd USENIX Security Symposium (USENIX Security 23)*, pp. 2205–2222, 2023.

[53] Y. Li, F. hossain shezan, B. wei, G. Wang, and Y. Tian, "Sok: Towards effective automated vulnerability repair," 2025.

[54] A. Smith and M. Patel, "Longitudinal studies of ai in safety-critical systems: Lessons for software security," *IEEE Transactions on Dependable and Secure Computing*, 2025.

[55] S. Shukla, H. Joshi, and R. Syed, "Security degradation in iterative ai code generation: A systematic analysis of the paradox," *arXiv preprint arXiv:2506.11022*, 2025.

[56] J. Kim, W. Choi, and B. Lee, "Prompt flow integrity to prevent privilege escalation in llm agents," in *Proceedings of the 2025 Conference on Machine Learning and Security*, 2025.

[57] Y. Xu, K. Zhang, M. Liu, and X. Chen, "Llm-assisted malware generation: Risks and defenses," in *Proceedings of the IEEE Symposium on Security and Privacy Workshops*, 2023.

[58] Z. Yang, Z. Zhao, C. Wang, J. Shi, D. Kim, D. Han, and D. Lo, "Gotcha! this model uses my code! evaluating membership leakage risks in code models," 2024.

[59] X. Zhou, M. Weyssow, R. Widyasari, T. Zhang, J. He, Y. Lyu, J. Chang, B. Zhang, D. Huang, and D. Lo, "Lessleak-bench: A first investigation of data leakage in llms across 83 software engineering benchmarks," 2025.

[60] M. Kozak, R. Zilouchian Moghaddam, and S. Sivaraman, "When developer aid becomes security debt: A systematic analysis of insecure behaviors in llm coding agents," *arXiv preprint arXiv:2507.09329*, 2025.

[61] M. Brundage, S. Avin, J. Wang, H. Belfield, G. Krueger, G. Hadfield, H. Khlaaf, J. Yang, H. Toner, R. Fong, T. Maharaj, P. W. Koh, S. Hooker, J. Leung, A. Trask, E. Bluemke, J. Lebensold, C. O'Keefe, M. Koren, T. Ryffel, J. Rubinovitz, T. Besiroglu, F. Carugati, J. Clark, P. Eckersley, S. de Haas, M. Johnson, B. Laurie, A. Ingerman, I. Krawczuk, A. Askell, R. Cammarota, A. Lohn, D. Krueger, C. Stix, P. Henderson, L. Graham, C. Prunkl, B. Martin, E. Seger, N. Zilberman, S. hÉigeartaigh, F. Kroeger, G. Sastry, R. Kagan, A. Weller, B. Tse, E. Barnes, A. Dafoe, P. Scharre, A. Herbert-Voss, M. Rasser, S. Sodhani, C. Flynn, T. K. Gilbert, L. Dyer, S. Khan, Y. Bengio, and M. Anderljung, "Toward trustworthy ai development: Mechanisms for supporting verifiable claims," 2020.

[62] A. Cuzzocrea, F. Furfaro, and D. Saccà, "Enabling olap in mobile environments via intelligent data cube compression techniques," *Journal of Intelligent Information Systems*, vol. 33, no. 2, pp. 95–143, 2009.

[63] A. Cuzzocrea, F. Furfaro, and D. Saccà, "Hand-olap: a system for delivering olap services on handheld devices," in *The Sixth International Symposium on Autonomous Decentralized Systems, 2003. ISADS 2003.*, pp. 80–87, IEEE, 2003.

[64] Y. Chen, Z. Ding, L. Alowain, X. Chen, and D. Wagner, "Diversevul: A new vulnerable source code dataset for deep learning based vulnerability detection," 2023.

Authorized licensed use limited to: Nara Institute of Science and Technology (NAIST). Downloaded on April 08,2026 at 13:32:24 UTC from IEEE Xplore. Restrictions apply.